LABSHEET-6

Q1. Handling Arithmetic Exception Write a Java program that accepts two integers from the user and divides the first number by the second number. Handle the situation where the user enters 0 as the denominator using try-catch. Concept: try, catch, Arithmetic Exception

```
import java.util.Scanner;


class ArithmeticExceptionDemo {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter first number: ");
      int a = sc.nextInt();


      System.out.print("Enter second number: ");
      int b = sc.nextInt();


      try {
         int result = a / b;
         System.out.println("Result = " + result);
      }
      catch (ArithmeticException e) {
         System.out.println("Cannot divide by zero!");
      }


      sc.close();
   }
}
```

OUTPUT-:

Enter first number: 10

Enter second number: 0

Cannot divide by zero!

Q2. Handling ArrayIndexOutOfBoundsException Create an integer array containing 5 elements. Ask the user to enter an array index and display the corresponding element. Handle the situation where the user enters an invalid index. Concept: ArrayIndexOutOfBoundsException

```
import java.util.Scanner;


class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      int arr[] = {10, 20, 30, 40, 50};


      System.out.print("Enter array index: ");
      int index = sc.nextInt();


      try {
         System.out.println("Element: " + arr[index]);
      }
      catch (ArrayIndexOutOfBoundsException e) {
         System.out.println("Invalid index! Please enter index from 0 to 4.");
      }


      sc.close();
   }
}
```

OUTPUT:-

Enter array index: 7

Invalid index! Please enter index from 0 to 4.

Q3. Handling NumberFormatException Write a Java program that stores a number as a String and converts it into an integer using Integer.parseInt(). Handle the exception if the string does not contain a valid integer. Concept: NumberFormatException

```
class Main {
  public static void main(String[] args) {

    String str = "123";

    try {
      int num = Integer.parseInt(str);
      System.out.println("Integer value: " + num);
    }
    catch (NumberFormatException e) {
      System.out.println("Invalid number format!");
    }
  }
}
```

OUTPUT-:

Invalid Number

If we change String str = "123"; to String str = "abc";, the output will be:

Invalid number format!

Q4. Multiple catch Blocks Write a Java program that: 1. Accepts two numbers from the user. 2. Performs division. 3. Creates an integer array containing five elements. 4. Accepts an index from the user. 5. Displays the corresponding array element. Handle the following exceptions using separate catch blocks: • ArithmeticException • ArrayIndexOutOfBoundsException • NumberFormatException Concept: Multiple catch blocks

```
import java.util.Scanner;
```

```
class Main {
 public static void main(String[] args) {
   Scanner sc = new Scanner(System.in);

   try {
     System.out.print("Enter first number: ");
     int a = Integer.parseInt(sc.next());

     System.out.print("Enter second number: ");
     int b = Integer.parseInt(sc.next());

     System.out.println("Division: " + (a / b));

     int arr[] = {10, 20, 30, 40, 50};

     System.out.print("Enter array index: ");
     int index = Integer.parseInt(sc.next());

     System.out.println("Array element: " + arr[index]);
   }
   catch (ArithmeticException e) {
     System.out.println("Cannot divide by zero!");
   }
   catch (ArrayIndexOutOfBoundsException e) {
     System.out.println("Invalid array index!");
   }
   catch (NumberFormatException e) {
```

```
        System.out.println("Please enter valid integers!");
      }

      sc.close();
   }
}
```

Output

Enter first number: 10

Enter second number: 2

Division: 5

Enter array index: 7

Invalid array index!

Q5. Using finally Write a Java program that accepts two numbers and performs division. Use a finally block to display: Program execution completed. The message must be displayed whether an exception occurs or not. Concept: finally

```
import java.util.Scanner;

class FinallyExample {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         System.out.print("Enter first number: ");
         int a = sc.nextInt();

         System.out.print("Enter second number: ");
         int b = sc.nextInt();
```

```
        int result = a / b;
        System.out.println("Division = " + result);
      }
      catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero.");
      }
      finally {
        System.out.println("Program execution completed.");
      }
    }
}
```

OUTPUT:-

Enter first number: 10

Enter second number: 2

Division = 5

Program execution completed.

Q6. Using throw Write a Java program that accepts a student's marks. If the marks are: • Less than 0, or • Greater than 100 explicitly generate an exception using the throw keyword and display an appropriate error message. Concept: throw

```
import java.util.Scanner;


class ThrowExample {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      System.out.print("Enter student's marks: ");
      int marks = sc.nextInt();
```

```
    try {
      if (marks < 0 || marks > 100) {
        throw new ArithmeticException("Invalid marks!");
      }

      System.out.println("Marks = " + marks);
    }
    catch (ArithmeticException e) {
      System.out.println("Error: " + e.getMessage());
    }
  }
}
```

OUTPUT:-

Enter student's marks: 120

Error: Invalid marks!

Q7. Using throws Create a method named checkAge(int age). If the age is less than 18, the method should generate an exception. Declare the exception using the throws keyword. Call the method from main() and handle the exception. Concept: throws

```
class ThrowsExample {

  static void checkAge(int age) throws Exception {
    if (age < 18) {
      throw new Exception("Age must be 18 or above.");
    }
    System.out.println("You are eligible.");
  }

  public static void main(String[] args) {
```

```
    try {
      checkAge(16);
    }
    catch (Exception e) {
      System.out.println("Error: " + e.getMessage());
    }
  }
}
```

OUTPUT:-

Error: Age must be 18 or above.

Q8. Nested try-catch Write a Java program containing a try-catch block inside another try block. • The inner try block should perform array access. • The outer try block should perform division. Handle both possible exceptions. Concept: Nested try-catch

```
import java.util.Scanner;

class NestedTryExample {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    try {
      System.out.print("Enter two numbers: ");
      int a = sc.nextInt();
      int b = sc.nextInt();

      int result = a / b;
      System.out.println("Division = " + result);

      try {
```

```
            int arr[] = {10, 20, 30, 40, 50};

            System.out.print("Enter array index: ");
            int index = sc.nextInt();

            System.out.println("Array element = " + arr[index]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index.");
        }
      }
      catch (ArithmeticException e) {
        System.out.println("Cannot divide by zero.");
      }
    }
}
```

OUTPUT:-

Enter two numbers: 10 2

Division = 5

Enter array index: 2

Array element = 30

Q9. Exception Propagation Create three methods: method1() method2() method3() method3() should generate an ArithmeticException. Allow the exception to propagate from method3() through method2() and method1() and finally handle it in main(). Concept: Exception propagation

```
class ExceptionPropagation {

   static void method3() {
      int a = 10 / 0;
```

```
  }

  static void method2() {
    method3();
  }

  static void method1() {
    method2();
  }

  public static void main(String[] args) {
    try {
      method1();
    }
    catch (ArithmeticException e) {
      System.out.println("Exception handled in main.");
    }
  }
}
```

OUTPUT:-

Exception handled in main.

Q10. User-Defined Exception Create a custom exception named InvalidMarksException. Accept marks from the user. If the marks are outside the range 0–100, throw the custom exception and display an appropriate error message. Concept: Custom exception, extends Exception, throw

```
import java.util.Scanner;

class InvalidMarksException extends Exception {
  InvalidMarksException(String message) {
```

```
        super(message);
    }
}

class CustomExceptionExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter student's marks: ");
            int marks = sc.nextInt();

            if (marks < 0 || marks > 100) {
                throw new InvalidMarksException("Marks must be between 0 and 100.");
            }

            System.out.println("Valid marks: " + marks);
        }
        catch (InvalidMarksException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

OUTPUT:-

Enter student's marks: 120

Error: Marks must be between 0 and 100.

Q11. Bank Withdrawal System Create a Java program for a simple bank withdrawal system. Accept: • Account balance • Withdrawal amount Apply the following rules: 1. Withdrawal amount cannot be

negative. 2. Withdrawal amount cannot be greater than the available balance. 3. If the withdrawal amount is greater than the balance, create and throw a custom exception named InsufficientBalanceException. 4. Handle invalid numeric input. 5. Use a finally block to display: Bank transaction completed. Concepts: Custom exception, throw, multiple catch, finally, NumberFormatException

```
import java.util.Scanner;


class InsufficientBalanceException extends Exception {
  InsufficientBalanceException(String message) {
    super(message);
  }
}


class BankWithdrawal {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);


    try {
      System.out.print("Enter account balance: ");
      double balance = Double.parseDouble(sc.nextLine());


      System.out.print("Enter withdrawal amount: ");
      double amount = Double.parseDouble(sc.nextLine());


      if (amount < 0) {
        throw new IllegalArgumentException("Withdrawal amount cannot be negative.");
      }


      if (amount > balance) {
```

```
            throw new InsufficientBalanceException("Insufficient balance!");
        }

        balance = balance - amount;
        System.out.println("Withdrawal successful.");
        System.out.println("Remaining balance: " + balance);
      }
      catch (NumberFormatException e) {
        System.out.println("Error: Please enter valid numbers.");
      }
      catch (InsufficientBalanceException e) {
        System.out.println("Error: " + e.getMessage());
      }
      catch (IllegalArgumentException e) {
        System.out.println("Error: " + e.getMessage());
      }
      finally {
        System.out.println("Bank transaction completed.");
      }
    }
}
```

OUTPUT:-

```
Enter account balance: 5000
Enter withdrawal amount: 2000
Withdrawal successful.
Remaining balance: 3000.0
Bank transaction completed.
```

Q12. Hospital Patient Age Validation Create a Java program for hospital patient registration. Accept: • Patient name • Patient age The valid age range is 0–120. Create a custom exception named InvalidPatientAgeException. If the age is outside the valid range, throw the custom exception. Also handle the situation where the user enters non-numeric input for age. Concepts: Custom exception, throw, catch, NumberFormatException

```
import java.util.Scanner;


class InvalidPatientAgeException extends Exception {
  InvalidPatientAgeException(String message) {
    super(message);
  }
}


class HospitalRegistration {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);


    try {
      System.out.print("Enter patient name: ");
      String name = sc.nextLine();


      System.out.print("Enter patient age: ");
      int age = Integer.parseInt(sc.nextLine());


      if (age < 0 || age > 120) {
        throw new InvalidPatientAgeException("Invalid patient age!");
      }


      System.out.println("Patient registered successfully.");
```

```
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
      }
      catch (InvalidPatientAgeException e) {
        System.out.println("Error: " + e.getMessage());
      }
      catch (NumberFormatException e) {
        System.out.println("Error: Please enter a valid numeric age.");
      }
    }
}
```

OUTPUT:-

Enter patient name: Varnika

Enter patient age: 150

Error: Invalid patient age!

Q13. Pharmaceutical Drug Dosage Validation Create a Java program for validating a patient's drug dosage. Accept: • Patient name • Drug name • Dosage in mg The valid dosage range is 1 mg–1000 mg. Create a custom exception named InvalidDosageException. If the dosage is less than or equal to 0, or greater than 1000, throw the custom exception. Also handle invalid numeric input and use a finally block to indicate that dosage validation is completed. Concepts: Custom exception, throw, try-catch, finally, NumberFormatException

```
import java.util.Scanner;


class InvalidDosageException extends Exception {
  InvalidDosageException(String message) {
    super(message);
  }
}
```

```
class DrugDosageValidation {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         System.out.print("Enter patient name: ");
         String name = sc.nextLine();

         System.out.print("Enter drug name: ");
         String drug = sc.nextLine();

         System.out.print("Enter dosage in mg: ");
         double dosage = Double.parseDouble(sc.nextLine());

         if (dosage <= 0 || dosage > 1000) {
            throw new InvalidDosageException("Invalid dosage! Range is 1-1000 mg.");
         }

         System.out.println("Patient Name: " + name);
         System.out.println("Drug Name: " + drug);
         System.out.println("Dosage: " + dosage + " mg");
         System.out.println("Dosage is valid.");

      }
      catch (InvalidDosageException e) {
         System.out.println("Error: " + e.getMessage());
      }
```

```
    catch (NumberFormatException e) {
      System.out.println("Error: Please enter a valid numeric dosage.");
    }
    finally {
      System.out.println("Dosage validation completed.");
    }
  }
}
```

OUTPUT:-

```
Enter patient name: Varnika
Enter drug name: Medicine
Enter dosage in mg: 500
Patient Name: Varnika
Drug Name: Medicine
Dosage: 500.0 mg
Dosage is valid.
Dosage validation completed.
```

Q14. Online Examination System Create a Java program for an online examination system. Accept a student's marks. Apply the following rules: 0–100 → Valid < 0 → Invalid > 100 → Invalid Create a custom exception named InvalidExamMarksException. The program should: 1. Accept marks. 2. Validate the marks. 3. Throw the custom exception if marks are invalid. 4. Display PASS if marks are 40 or above. 5. Display FAIL if marks are below 40. 6. Handle invalid numeric input. 7. Use finally to display: Exam evaluation completed. Concepts: Custom exception, throw, catch, finally, NumberFormatException

```
import java.util.Scanner;

class InvalidExamMarksException extends Exception {
  InvalidExamMarksException(String message) {
    super(message);
```

```
    }
}

class OnlineExamination {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter student's marks: ");
            int marks = Integer.parseInt(sc.nextLine());

            if (marks < 0 || marks > 100) {
                throw new InvalidExamMarksException("Invalid marks! Enter marks between 0 and 100.");
            }

            if (marks >= 40) {
                System.out.println("PASS");
            } else {
                System.out.println("FAIL");
            }
        }
        catch (InvalidExamMarksException e) {
            System.out.println("Error: " + e.getMessage());
        }
        catch (NumberFormatException e) {
            System.out.println("Error: Please enter valid numeric marks.");
        }
```

```
    finally {
      System.out.println("Exam evaluation completed.");
    }
  }
}
```

OUTPUT:-

Enter student's marks: 75

PASS

Exam evaluation completed.

Q15. Pharmacy Inventory System Create a Java program for a pharmacy inventory system. Accept: • Medicine name • Available quantity • Required quantity Apply the following rules: 1. Quantity cannot be negative. 2. Required quantity cannot be greater than available quantity. 3. If any quantity is negative, throw InvalidQuantityException. 4. If required quantity is greater than available quantity, throw InsufficientMedicineStockException. 5. Handle invalid numeric input. 6. Use multiple catch blocks. 7. Use a finally block to indicate that the inventory transaction is completed. Concepts: Multiple custom exceptions, throw, multiple catch, finally, NumberFormatException

```
import java.util.Scanner;


class InvalidQuantityException extends Exception {
  InvalidQuantityException(String message) {
    super(message);
  }
}


class InsufficientMedicineStockException extends Exception {
  InsufficientMedicineStockException(String message) {
    super(message);
  }
}
```

```
class PharmacyInventory {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter medicine name: ");
            String name = sc.nextLine();

            System.out.print("Enter available quantity: ");
            int available = Integer.parseInt(sc.nextLine());

            System.out.print("Enter required quantity: ");
            int required = Integer.parseInt(sc.nextLine());

            if (available < 0 || required < 0) {
                throw new InvalidQuantityException("Quantity cannot be negative.");
            }

            if (required > available) {
                throw new InsufficientMedicineStockException("Insufficient medicine stock!");
            }

            System.out.println("Medicine Name: " + name);
            System.out.println("Medicine issued successfully.");
            System.out.println("Remaining quantity: " + (available - required));
        }
```

```
        catch (InvalidQuantityException e) {
            System.out.println("Error: " + e.getMessage());
        }
        catch (InsufficientMedicineStockException e) {
            System.out.println("Error: " + e.getMessage());
        }
        catch (NumberFormatException e) {
            System.out.println("Error: Please enter valid numeric quantities.");
        }
        finally {
            System.out.println("Inventory transaction completed.");
        }
    }
}
```

OUTPUT:-

```
Enter medicine name: Paracetamol
Enter available quantity: 100
Enter required quantity: 30
Medicine Name: Paracetamol
Medicine issued successfully.
Remaining quantity: 70
Inventory transaction completed.
```